



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)

Re-Accredited by NAAC with 'A++' Grade

Subject: Programming with Java

Name: Aman S. Choudhary

PRN: 25070122033

Div: CSE A1

Batch: 2025-29

ASSIGNMENT No: 3

Title

Develop a Java application demonstrating method overloading and the use of static variables, methods in class design.

Problem Statement

Develop a Java application demonstrating method overloading and the use of static variables, methods in class design.

Learning Objectives

To implement method overloading and use static variables and methods in Java programs.

Theory

Method overloading allows multiple methods to have the same name with different parameter lists, enabling compile-time polymorphism. The static keyword is used for variables and methods that belong to the class instead of individual objects. Static members share common data among all objects and reduce memory usage. These concepts improve code flexibility, readability, and efficient resource management.

Outcome

Method overloading was successfully implemented to perform addition and billing operations using different parameter types and counts, and static variables were used to keep a shared, running count of calculations and orders across all method calls.

Conclusion

This experiment helped us understand how method overloading lets one method name behave differently depending on its parameters, giving programs a clean and readable interface. It also showed how static variables and methods belong to the class as a whole rather than any single object, making them ideal for tracking shared information such as a calculation count or an order count. Together, these features make Java code more flexible, reusable, and memory-efficient, which are core principles of good class design.

Exercise

1. Calculator Program

A calculator using overloaded add() methods for two integers, three integers, and two decimals, with a static variable counting how many calculations were performed.

```
1  //prog for method overloading
2  public class Calc{
3      public static int add(int a, int b){
4          //System.out.println("Two int parameters:");
5          return a+b;
6      }
7      public static double add(double a, double b){
8          //System.out.println("Two db parameters:");
9          return a+b;
10     }
11     public static int add(int a, int b, int c){
12         //System.out.println("Three int parameters:");
13         return a+b+c;
14     }
15     public static void main(String[] args) {
16         int intResult1 = Calc.add(a: 5, b: 10);
17         System.out.println("Result: " + intResult1);
18
19         double doubleResult = Calc.add(a: 5.5, b: 4.3);
20         System.out.println("Result: " + doubleResult);
21
22         int intResult2 = Calc.add(a: 10, b: 20, c: 30);
23         System.out.println("Result: " + intResult2);
24     }
25 }
```

Sample Output:

```
PS C:\Users\Admin\JAVA> &
083' '--enable-preview' '-X
c34b2d7d08b19\redhat.java\j
Result: 15
Result: 9.8
Result: 60
```

2. Restaurant Billing Application

A billing system using overloaded calculateBill() methods for dine-in (with tax), takeaway (with packaging charge), and delivery (with delivery charge) orders, with a static variable tracking the total number of orders.

```
1  public class RestaurantBilling {
2      static int totalOrders = 0;
3      static final double SERVICE_TAX = 0.05;
4      static final double DELIVERY_CHARGE = 30.0;
5      // overload 1: dine-in bill -> adds service tax
6      static double calculateBill(double amount) {
7          totalOrders++;
8          return amount + (amount * SERVICE_TAX);
9      }
10     // overload 2: takeaway bill -> adds a packaging charge
11     static double calculateBill(double amount, double packagingCharge) {
12         totalOrders++;
13         return amount + packagingCharge;
14     }
15     // overload 3: delivery bill -> adds a fixed delivery charge
16     static double calculateBill(double amount, boolean isDelivery) {
17         totalOrders++;
18         return isDelivery ? amount + DELIVERY_CHARGE : amount;
19     }
20     Run | Debug
21     public static void main(String[] args) {
22         double dineIn = calculateBill(amount: 500.0);
23         System.out.println("Dine-in bill: Rs. " + dineIn);
24
25         double takeaway = calculateBill(amount: 300.0, packagingCharge: 10.0);
26         System.out.println("Takeaway bill: Rs. " + takeaway);
27
28         double delivery = calculateBill(amount: 400.0, isDelivery: true);
29         System.out.println("Delivery bill: Rs. " + delivery);
30
31         System.out.println("Total orders placed: " + totalOrders);
32     }
```

Sample Output:

```
Dine-in bill: Rs. 525.0
Takeaway bill: Rs. 310.0
Delivery bill: Rs. 430.0
Total orders placed: 3
```
